

소프트웨어와 인공지능 이해 및 응용 중간 압축본

Week 1. Week1-1/1-2: 컴퓨팅 및 소프트웨어의 역사

핵심 흐름

- **single-purpose machine**: 같은 동작만 반복, 특수 목적
- **Turing model**: program이 추가되어 범용 목적 작업 가능
- **Turing machine**: 테이프의 기호를 읽고/쓰며 규칙대로 계산하는 이론적 기계
- **Universal Turing Machine**: 임의의 튜링 머신 시뮬레이션 가능 -> 현대 컴퓨터의 모태

컴퓨터 3요소

- **hardware / data / software**
- software = 하드웨어가 데이터를 처리하도록 지시하는 프로그램

역사 포인트

- **Babbage**: Difference Engine, Analytical Engine
- **Hollerith**: punched card 기반 programmer machine
- **ENIAC**: 초기 범용 전자 컴퓨터

한 줄 정리

- 현대 컴퓨터의 본질 = **프로그램을 바꿔 다양한 작업을 하는 범용성**

Week 2. Week2-1/2-2: 숫자 데이터의 표현 및 연산

기수법

- **base(radix)**: 자릿값의 기준
- positional number system: 같은 기호라도 **자리**에 따라 값이 달라짐

정수 표현

- **unsigned integer**
 - 음수 X
 - n-bit 범위: $0 \sim 2^n - 1$
- **overflow**
 - 자료형 범위를 넘어서 표현 불가능한 상태

signed integer

- **sign-and-magnitude**
 - 맨 왼쪽 비트 = 부호
 - 단점: '+0', '-0' 두 개 존재

- **two's complement**
 - 음수 정수의 표준 표현
 - 구하는 법: **1의 보수 + 1**
 - n-bit 범위: $-2^{(n-1)} \sim 2^{(n-1)} - 1$
 - 장점: 0 표현 1개, 연산 편함

실수 표현

- **floating point**
 - 구성: **sign / exponent / mantissa**
 - 매우 큰 수/작은 수 표현 가능

시험용 포인트

- 음수 정수 표준 = 'two's complement'
- 실수 표준 = 'floating point'

Week 3. Week3-1/3-2: 다양한 데이터의 표현 및 저장

문자열

- **ASCII**
 - 7-bit 코드
 - 영문/기본 특수문자 중심
- **Unicode**
 - 현대 표준 문자 표현
 - 다양한 언어 지원

음성

- 아날로그 -> 디지털 변환 필요
- **sampling**: 일정 간격으로 추출
- **quantization**: 값을 근사한 이산값으로 변환

이미지

- **bitmap**: 픽셀 기반, 해상도/색 깊이에 영향 받음
- **vector graphics**: 도형/수식 기반, 확대/축소에 강함

비디오

- 기본적으로 **frames of images**

압축

- 음성/동영상은 압축 중요
- 이미지도 압축 많이 활용

한 줄 비교

- 'ASCII vs Unicode' = 기본 영문 코드 vs 다국어 표준
- 'bitmap vs vector' = 픽셀 기반 vs 도형 기반

Week 4. Week4-1/4-2/4-3: 컴퓨터 구조와 운영체제

하드웨어 3파트

- CPU
- Main memory
- I/O subsystem

CPU 구성

- ALU: 산술/논리/시프트 연산
- register
 - CPU 내부 저장공간
 - 가장 빠름
- control unit: 각 서브시스템 제어

메모리

- 주소('address')를 통해 접근
- word: 메모리 읽기/쓰기의 기본 단위
- address space: 표현 가능한 주소 범위

주소공간

- word = 1 byte일 때
- 32-bit address -> 2^{32} byte = 4GB
- 더 큰 메모리를 쓰려면 64-bit 주소체계 필요

메모리 계층

- register > cache > main memory
- 속도 빠름 -> 느림, CPU 가까움 -> 멀

I/O subsystem

- non-storage device: keyboard, mouse, monitor, printer 등
- storage device: magnetic disk, optical storage, flash memory

운영체제(OS)

- 사용자와 하드웨어 사이의 인터페이스
- 다른 프로그램 실행을 위한 기반
- 컴퓨터 자원 관리
- 예: Windows, Linux, Mac OS, Android, iOS

bootstrap process

- OS도 메모리에 올라와야 실행 가능
- 일반 흐름:
 1. ROM에서 작은 bootstrap program 로드
 2. bootstrap program이 OS를 메모리로 로드
 3. 실행 흐름을 OS로 넘김

시험용 포인트

- 'register'가 가장 빠름
- '32-bit OS -> 4GB'
- OS 역할 = 인터페이스 + 실행 기반 + 자원 관리

Week 5. Week5-1/5-2: 알고리즘과 문제 해결, 프로그래밍 언어

알고리즘

- 문제를 해결하기 위한 명확한 단계적 절차
- 핵심 구성 요소 3개
 - sequence
 - decision(if/else)
 - repetition(loop)

표현 방식

- UML
 - 모델링 언어
 - 알고리즘 + 클래스 구조 + 함수 관계 표현 가능
- pseudocode
 - 특정 언어에 묶이지 않는 알고리즘 표현
 - 자연어와 프로그래밍 언어의 중간

selection sort

- 정렬된 영역 / 정렬되지 않은 영역으로 나눔
- 매 단계마다 최솟값을 찾아 정렬된 영역 뒤에 놓음
- 정렬된 영역이 점점 커짐

프로그래밍 언어

- low-level : 기계 친화적
 - 예: machine language, assembly
- high-level : 인간 친화적
 - 복잡한 프로그램 작성 쉬움

compiler vs interpreter

- compiler
 - 전체 코드를 먼저 기계어로 번역 후 실행
 - 예: C++
- interpreter
 - 실행하면서 한 줄씩 번역
 - 예: Python
- 인터프리터 기반 언어를 script language라고도 함

한 줄 비교

- 'compiler' = 미리 번역
- 'interpreter' = 실행 중 번역

Week 6. Week6-1/6-2: 컴퓨터 네트워크와 인터넷

네트워크

- 통신 가능한 기기들의 상호 연결
- 구성 요소
 - **host(end system)**: 실제 작업 수행
 - **connecting device**: 통신 중개

프로토콜

- 통신 규칙
- 복잡해지면 **layering** 사용
- 같은 계층끼리 논리적으로 연결되었다고 보는 개념 = **logical connection**

TCP/IP

- 현대 인터넷의 대표적 프로토콜 스위트
- 보통 **4계층/5계층**으로 설명

연결 장비

- **hub**
 - 연결된 모두에게 정보 전파
 - 구분 못함
- **switch**
 - LAN 수준
 - 여러 기기를 **구분하여** 연결
- **router**
 - 네트워크 간 연결
 - 패킷의 경로(route) 지정 -> **routing**

packet

- 데이터를 쪼개어 보낸 각 조각

addressing

- **Port / IP address / MAC address**
- 시험 포인트:
 - **transport layer** -> **port**
 - **network layer** -> **IP**
 - **network access/data link** 쪽 -> **MAC**

application layer protocol

- **HTTP**
- **FTP**
- **TELNET**
- **SSH**
- **DNS**
- **Electronic mail**

WWW vs Internet

- **Internet**: 더 큰 개념, 전 세계 네트워크 연결
- **WWW(Web)**: 인터넷 위에서 hypertext로 연결된 웹페이지 구조

URL 구성

- **protocol / host / port / path**

자주 나오는 포트

- FTP: '20, 21'
- SSH: '22'
- HTTP: '80'
- HTTPS: '443'

한 줄 비교

- 'hub vs switch vs router'
 - 구분 없이 뿌림 / LAN에서 구분 연결 / 네트워크 간 경로 지정

Week 7. Week7-1/7-2/7-3: 인공지능의 개념과 역사

최근 AI 활약 예시

- 딥블루(체스)
- 왓슨(퀴즈쇼)
- 알파고(바둑)
- ChatGPT / DALL-E

strong AI vs weak AI

- **strong AI**
 - 자의식 있음
 - 일반적 문제도 스스로 해결 가능
- **weak AI**
 - 자의식 없음
 - 특정 영역 문제 해결
 - 예: 알파고

인간 vs AI

- AI: 빠른 계산, 논리적 추론, 학습
- 인간: 창의적 문제 해결
- 결론: **상호보완적**

AI 정의 4관점

- **Thinking Humanly**
- **Thinking Rationally**
- **Acting Humanly**
- **Acting Rationally**

ML / DL 관계

- **AI > Machine Learning > Deep Learning**
- DL은 ML + 인공신경망 + 대규모 데이터/연산

튜링 테스트

- 기계와 인간을 **구별할 수 없으면** AI로 보자

ELIZA

- 특정 키워드 기반 규칙으로 답변 생성
- 사람처럼 보일 수 있어도 **이해**를 보장하진 않음

중국인 방

- Searle의 사고실험
- 기호 조작만으로는 **이해**라고 볼 수 없다고 주장
- 결론: **튜링 테스트 통과 = 진짜 사고 입증 X**

튜링 테스트의 문제점

- 인간적 행동 = 지적 행동은 아님
- 지적 행동 = 인간적 행동도 아님
- 기준이 오래되었다는 비판 존재

AI 역사 흐름

- **탐색 중심 AI** -> 조합 폭발
- **첫 번째 AI 겨울** -> 연산/데이터 부족
- **전문가 시스템** -> 특정 분야 지식 집중
- **두 번째 AI 겨울** -> 제한된 분야만 강함
- **딥러닝 부활** -> ML + 신경망 결합
- **초거대 AI** -> 대용량 데이터 + 강력한 인프라, 예: GPT

전문가 시스템

- 모든 문제 대신 **특정 분야** 해결에 집중
- 대표 예: **DENDRAL**
- 한계: 일반 목적 질의응답, 규칙 계속 추가되는 상황에 약함

AI 응용 분야

- 자율주행
- 추천 시스템
- 챗봇 / 가상 비서
- 생성형 AI
- 의료 AI
- 신약 개발(예: AlphaFold)

한 줄 비교

- 'Turing test vs Chinese room'
 - 구별 불가 여부 / 이해 부재 비판

시험 직전 체크리스트

- 'two's complement' 왜 쓰는지
- '32-bit OS -> 4GB' 이유
- 'register > cache > main memory'
- 'OS 역할 3개'
- 'sequence / decision / repetition'
- 'compiler vs interpreter'
- 'switch vs router'
- 'port / IP / MAC'
- 'WWW vs Internet'
- 'strong AI vs weak AI'
- '튜링 테스트 vs 중국인 방'
- '탐색 -> AI 겨울 -> 전문가 시스템 -> AI 겨울 -> 딥러닝 부활'